

SECTION 1

Mathematical & Statistical Foundations

The non-negotiable bedrock of deep learning

Deep learning is applied mathematics. You cannot truly understand why a technique works — or why it fails — without grasping the underlying mathematics. This section is not a full textbook; it focuses specifically on the mathematical structures that appear constantly in neural network theory and practice.

1.1 Linear Algebra for Deep Learning

Every piece of data in a neural network is a tensor — a generalization of scalars, vectors, and matrices to arbitrary dimensions. Understanding how these objects transform is essential.

Vectors and Matrices

A vector $x \in \mathbb{R}^n$ represents a point in n -dimensional space. In DL, it could be a flattened image, a token embedding, or a sample of audio. A matrix $W \in \mathbb{R}^{m \times n}$ represents a linear transformation from $\mathbb{R}^n \rightarrow \mathbb{R}^m$. The matrix-vector product Wx is the core operation of every dense layer.

Σ
MATH

Linear Layer: $y = Wx + b$, where $W \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, $b \in \mathbb{R}^m$. This is a linear map composed with translation.

Tensors

A tensor is a multi-dimensional array. A 4D tensor of shape (N, C, H, W) represents a mini-batch of N images, each with C channels and spatial dimensions $H \times W$. Tensor operations (element-wise, broadcasting, reductions, reshapes) are the vocabulary of deep learning code.

Key Operations

- **Dot product:** $x \cdot y = \sum x_i y_i$ — measures alignment/similarity. Fundamental to attention.
- **Matrix multiplication:** AB — composes transformations. $O(n^3)$ naive, highly optimized in practice.
- **Transpose:** A^T — swaps rows and columns. Appears in gradient derivations constantly.
- **Eigendecomposition:** $A = Q\Lambda Q^T$ — reveals the stretching directions of a linear map. Connects to optimization landscape analysis.
- **SVD:** $A = U\Sigma V^T$ — any matrix factors into rotation $\times$ scaling $\times$ rotation. Used in compression, LoRA, analysis.

i
INFO

Why it matters: The entire forward pass of a neural network is a sequence of tensor operations. Backpropagation is computing gradients through those operations using the chain rule on matrix calculus.

1.2 Calculus — Differentiation & Gradients

Deep learning training is gradient-based optimization. Understanding derivatives is not optional.

Derivatives and Gradients

The derivative $f'(x)$ gives the rate of change of f at x . For a scalar function of a vector, the gradient $\nabla f(x) \in \mathbb{R}^n$ is the vector of partial derivatives — it points in the direction of steepest ascent.

Σ
MATH

Gradient: $\nabla f(x) = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]^T$. Gradient descent updates $x \leftarrow x - \eta \nabla f(x)$ to minimize f .

Chain Rule

The chain rule is backpropagation. If $y = f(g(x))$, then $dy/dx = (dy/dg)(dg/dx)$. For vectors: if $y = f(u)$, $u = g(x)$, then $\partial y / \partial x = (\partial y / \partial u)(\partial u / \partial x)$ — a Jacobian product.

★
KEY

Backpropagation is just the chain rule applied systematically from the loss backward through every operation in the computational graph. Nothing more.

Jacobians and Hessians

- **Jacobian $J \in \mathbb{R}^{m \times n}$** : matrix of all partial derivatives $\partial y_i / \partial x_j$. Generalizes derivative to vector-valued functions.
- **Hessian $H \in \mathbb{R}^{n \times n}$** : matrix of second-order partial derivatives. Describes local curvature. Expensive to compute but informs second-order optimizers.

1.3 Probability & Statistics

Neural networks learn probability distributions — explicitly in generative models, implicitly in discriminative ones.

Core Concepts

- **Probability distribution**: assigns likelihoods to outcomes. Key families: Gaussian (continuous, symmetric), Bernoulli (binary), Categorical (multi-class), Dirichlet (distributions over distributions).
- **Expectation**: $E[X] = \sum x p(x)$. The average under the distribution. Loss functions are expected costs.
- **Variance**: $E[(X - \mu)^2]$. Measures spread. High variance = unstable training signals.
- **Conditional probability**: $P(A|B) = P(A \cap B) / P(B)$. The foundation of generative modeling.
- **Bayes' theorem**: $P(\theta|D) = P(D|\theta)P(\theta) / P(D)$. Posterior = likelihood $\times$ prior / evidence.

Maximum Likelihood Estimation (MLE)

Most neural networks are trained by MLE: find parameters θ that maximize $P(\text{data} \mid \theta)$. Taking logs turns products to sums (numerical stability), and minimizing negative log-likelihood is equivalent to minimizing cross-entropy loss.

Σ
MATH

MLE: $\theta^* = \operatorname{argmax}_{\theta} \sum_i \log P(y_i \mid x_i, \theta)$. For Gaussian output, this equals minimizing MSE. For categorical output, this equals minimizing cross-entropy.

1.4 Information Theory

Information theory gives us the right language to talk about what a model learns and how well it has learned it.

Entropy

Entropy $H(X) = -\sum p(x) \log p(x)$ measures the average surprise (information content) of a distribution. High entropy = high uncertainty. A uniform distribution over 1000 classes has maximum entropy; a distribution peaked on one class has near-zero entropy.

Cross-Entropy

The cross-entropy $H(p, q) = -\sum p(x) \log q(x)$ measures how many bits are needed to encode events from distribution p using code designed for distribution q . When p is the true label distribution and q is the model's prediction, minimizing cross-entropy loss IS maximum likelihood training.

Σ
MATH

Cross-entropy loss: $L = -\sum_i y_i \log(\hat{y}_i)$. For one-hot labels y , reduces to $L = -\log(\hat{y}_{\text{true_class}})$.

KL Divergence

$KL(p \parallel q) = \sum p(x) \log(p(x)/q(x))$ measures how much information is lost when q approximates p . Always ≥ 0 . Not symmetric. Appears in VAE training (latent regularization), policy optimization in RL, and Bayesian inference.

NOTE

$KL(p \parallel q) \neq KL(q \parallel p)$. $KL(p \parallel q)$ penalizes placing zero probability where p is non-zero (zero-avoiding). $KL(q \parallel p)$ penalizes placing mass where p is zero (zero-forcing). This asymmetry drives different behaviors in generative models.